

Public GitHub repository containing code, animation and figures:

<https://github.com/CS23BTECH11028/cs23btech11028-mas-assignment3>

(Note: link to be updated after pushing. The repo folder in this submission mirrors what was pushed to GitHub.)

AI usage declaration. I used Claude to help me understand the concepts taught in the lecture material, namely the formation control law from the MAS Algorithms lecture, the distributed gradient descent (DGD) update from the Distributed Learning lecture, the consensus ADMM template from the ADMM lecture and the Helper notes, and the soft thresholding proximal operator. I also used it to speed up boiler plate Python (matplotlib animation setup, capped simplex projection routine). All derivations, algorithm choices, parameter values, and code checks were reviewed and rewritten by me. Any mathematics not covered in the notes (specifically the Metropolis-Hastings weight formula for the mixing matrix W in Problem 2, and the LP relaxation plus Hungarian based rounding in Problem 3) is explicitly declared in the corresponding section.

Problem 1

Fix $N = 20$. Generate a connected Erdos-Renyi graph. Assume random initial positions for all N agents in $\mathbb{R}^2$. Using formation control, display my name in capital letters in $\mathbb{R}^2$, letter by letter. The problem statement says for people with long names to use the short call name (the example given is that Manoj Kumar Sahu simulates just MANOJ). My call name is Krishna, so the sequence I have to display is $K \rightarrow R \rightarrow I \rightarrow S \rightarrow H \rightarrow N \rightarrow A$. Record the whole sequence as an animation, host it on a public GitHub repo, and submit the link.

Solution:

Idea. I use the classical distributed formation control law from Algorithm 2 of the MAS Algorithms lecture (page 7). Each agent has single integrator dynamics $\dot{p}_i = u_i$, and the discrete time update with step Δt is

$$u_i[k] = \sum_{j \in \mathcal{N}_i} a_{ij} [(p_j[k] - p_i[k]) - (r_j - r_i)], \quad p_i[k+1] = p_i[k] + \Delta t \cdot u_i[k].$$

The offsets $r_i \in \mathbb{R}^2$ describe the desired shape. If the graph is connected and Δt is small enough, this drives $p_i - p_j \rightarrow r_i - r_j$ for all i, j , so the agents lock into the target shape.

How the 20 agents draw a letter. For each capital letter I define a poly-line inside $[0, 1] \times [0, 1]$ consisting of a few straight segments and (for R , S and J) a few arcs. The poly-line is arc-length parameterised and I sample $N = 20$ points equally spaced along the total length. These 20 points become the offsets $r_1, \dots, r_{20}$ for that letter. Agent i is always mapped to the i -th sample position, so the correspondence is stable letter-to-letter which is what gives smooth transitions.

Graph. I use the same Erdos-Renyi construction that I used in Assignment 1. I resample until the graph is connected. Weights are $a_{ij} = 1$ if (i, j) is an edge and 0 otherwise (unweighted).

Parameters.

- $N = 20$, edge probability $p = 0.3$, seed = 7.
- Step $\Delta t = 0.05$, iterations per letter = 300.
- Sequence to draw: K R I S H N A (7 letters).

Graph. Figure 1 shows the generated communication graph.

Erdos-Renyi graph, $N=20$, $p=0.3$, connected

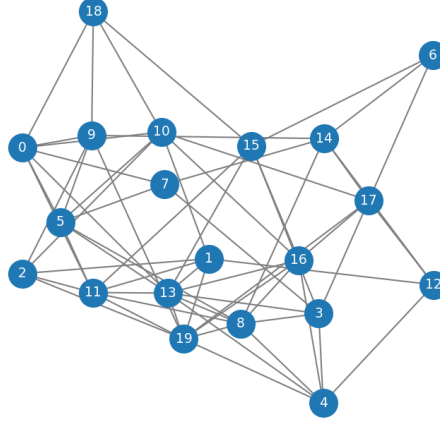


Figure 1: Erdos-Renyi communication graph with $N = 20$, $p = 0.3$, verified connected.

Animation. The full sequence of formations was recorded as an animated GIF; see the file `krishna_formation.gif` inside the `p1_formation/` folder in the GitHub repo. Figure 2 shows all seven letters captured at the end of their formation phase.

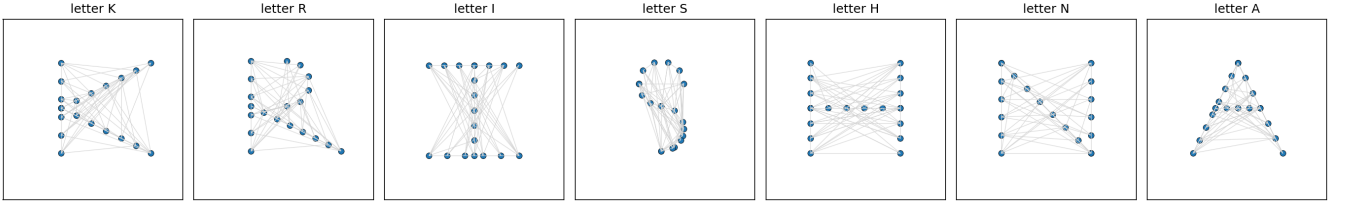


Figure 2: Still shots of every letter in the sequence K R I S H N A (end of each formation phase).

Formation error. Figure 3 plots $\max_i \|p_i - r_i\|$ across time. Every dashed vertical line marks the moment the target changes to the next letter. Inside each phase the error decays rapidly and settles to a small residual before the next letter is loaded.

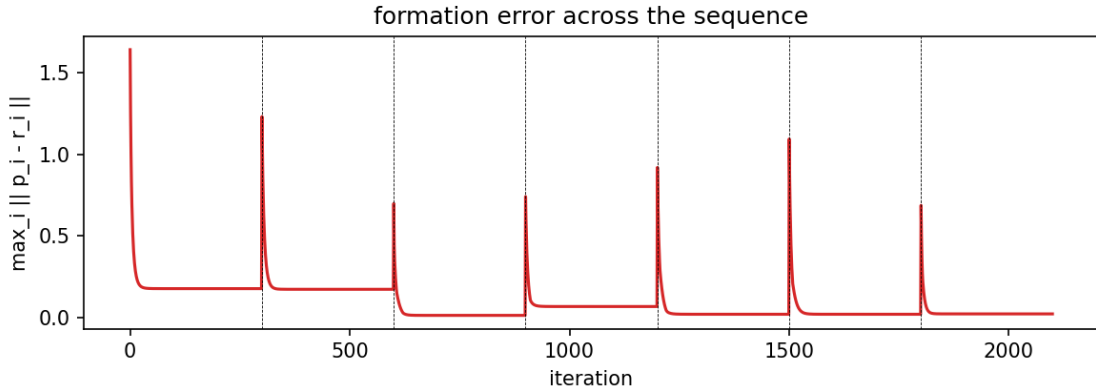


Figure 3: Max formation error $\max_i \|p_i - r_i\|$ across the whole sequence. Dashed vertical lines are the letter boundaries.

Observation. The formation controller is very robust here. Even though we ask for 7 back to back reshapes of the swarm, every letter is recognised at the end of its phase. Agents that were on the "wrong side" of the previous letter's shape smoothly cross over during the transition because the algorithm only

cares about the relative offset $r_j - r_i$, not about the absolute location.

Attached code.

```
1  """
2  Problem 1: Formation Control to spell KRISHNA TEJA with N=20 agents.
3
4  Reference:
5  - MAS Algorithms Lecture, Algorithm 2 (Distributed Formation Control), page 7.
6     $u_i[k] = \sum_{j \in N_i} a_{ij} [(p_j - p_i) - (r_j - r_i)]$ 
7     $p_i[k+1] = p_i[k] + dt * u_i[k]$ 
8
9  Graph: Erdos-Renyi  $G(N, p)$ , resampled until connected.
10 Author: Krishna Teja (CS23BTECH11028)
11 """
12
13 import os
14 import numpy as np
15 import matplotlib.pyplot as plt
16 import matplotlib.animation as animation
17 import networkx as nx
18
19 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
20
21 # ----- graph -----
22 def connected_er_graph(N, p, seed):
23     rng = np.random.default_rng(seed)
24     for _ in range(500):
25         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
26         if nx.is_connected(G):
27             return G
28     raise RuntimeError("could not get a connected ER graph")
29
30 # ----- letter shapes -----
31 def _line(a, b, n=30):
32     t = np.linspace(0, 1, n)
33     return np.stack([a[0] + t*(b[0]-a[0]), a[1] + t*(b[1]-a[1])], axis=1)
34
35 def _arc(c, r, a0, a1, n=40):
36     a = np.linspace(a0, a1, n)
37     return np.stack([c[0] + r*np.cos(a), c[1] + r*np.sin(a)], axis=1)
38
39 def letter_strokes(L):
40     """Return a list of strokes for letter L. Each stroke is a Kx2 poly-line
41     drawn inside [0,1]x[0,1]. Sampling later gives each stroke a share of
42     N proportional to its arc length (with a floor of 3 per stroke), so
43     short strokes like the middle bar of H still get enough points."""
44     if L == 'K':
45         return [_line((0,0),(0,1)),
46                 _line((0,0.5),(1,1)),
47                 _line((0,0.5),(1,0))]
48     if L == 'R':
49         return [_line((0,0),(0,1)),
50                 _arc((0.4,0.75), 0.25, np.pi/2, -np.pi/2),
51                 _line((0,0.5),(1,0))]
52     if L == 'I':
53         return [_line((0,1),(1,1)),
54                 _line((0.5,1),(0.5,0)),
55                 _line((0,0),(1,0))]
56     if L == 'S':
57         return [_arc((0.5,0.75), 0.25, 0, np.pi),
58                 _arc((0.5,0.75), 0.25, np.pi, 3*np.pi/2),
59                 _arc((0.5,0.25), 0.25, np.pi/2, -np.pi/2),
60                 _arc((0.5,0.25), 0.25, -np.pi/2, 0)]
61     if L == 'H':
62         return [_line((0,0),(0,1)),
```

```

63         _line((1,0),(1,1)),
64         _line((0,0.5),(1,0.5))]
65     if L == 'N':
66         return [_line((0,0),(0,1)),
67                 _line((0,1),(1,0)),
68                 _line((1,0),(1,1))]
69     if L == 'A':
70         return [_line((0,0),(0.5,1)),
71                 _line((0.5,1),(1,0)),
72                 _line((0.25,0.5),(0.75,0.5))]
73     if L == 'T':
74         return [_line((0,1),(1,1)),
75                 _line((0.5,1),(0.5,0))]
76     if L == 'E':
77         return [_line((0,0),(0,1)),
78                 _line((0,1),(1,1)),
79                 _line((0,0.5),(0.7,0.5)),
80                 _line((0,0),(1,0))]
81     if L == 'J':
82         return [_line((0,1),(1,1)),
83                 _line((1,1),(1,0.2)),
84                 _arc((0.5,0.2), 0.5, 0, -np.pi)]
85     raise ValueError(L)
86
87 def _sample_stroke_uniform(poly, n):
88     """Equally spaced n points along a single poly-line (arc-length param.)."""
89     d = np.linalg.norm(np.diff(poly, axis=0), axis=1)
90     s = np.concatenate([[0.0], np.cumsum(d)])
91     total = s[-1]
92     ts = np.linspace(0, total, n)
93     out = np.zeros((n, 2))
94     for k, t in enumerate(ts):
95         idx = np.searchsorted(s, t)
96         idx = min(max(idx, 1), len(poly)-1)
97         alpha = (t - s[idx-1]) / max(s[idx] - s[idx-1], 1e-12)
98         out[k] = poly[idx-1] + alpha * (poly[idx] - poly[idx-1])
99     return out
100
101 def sample_letter(strokes, N, floor_per_stroke=3):
102     """Distribute N samples over the strokes proportional to arc length,
103         but give every stroke at least `floor_per_stroke` points so that
104         short strokes (like the middle bar of H) are not under-represented."""
105     S = len(strokes)
106     if S * floor_per_stroke > N:
107         floor_per_stroke = max(1, N // S)
108     lengths = np.array([np.linalg.norm(np.diff(p, axis=0), axis=1).sum()
109                         for p in strokes])
110     remaining = N - S * floor_per_stroke
111     # proportional share of the remaining points by length
112     prop = lengths / lengths.sum()
113     extra = np.floor(prop * remaining).astype(int)
114     # hand out the leftovers (from rounding) to the longest strokes
115     leftover = remaining - extra.sum()
116     order = np.argsort(-lengths)
117     for k in range(leftover):
118         extra[order[k % S]] += 1
119     counts = extra + floor_per_stroke
120     assert counts.sum() == N
121     parts = [_sample_stroke_uniform(strokes[i], counts[i]) for i in range(S)]
122     return np.vstack(parts)
123
124 # ----- main sim -----
125 def main():
126     np.random.seed(7)
127     N = 20

```

```

128 p_edge = 0.3
129 G = connected_er_graph(N, p_edge, seed=7)
130 A = nx.to_numpy_array(G) # unweighted adjacency
131
132 name = "KRISHNA" # first name only, as instructed (Manoj example)
133
134 # per-letter targets, centered in a 1.2 x 1.2 box around origin
135 letters_pts = []
136 for L in name:
137     strokes = letter_strokes(L)
138     pts = sample_letter(strokes, N, floor_per_stroke=4) * 1.2 - np.array([0.6, 0.6])
139     letters_pts.append(pts)
140
141 # random initial positions in a big box
142 P = np.random.uniform(-1.6, 1.6, size=(N, 2))
143
144 dt = 0.05
145 iters_per_letter = 300
146 frames = [P.copy()]
147 letter_boundaries = [0] # frame index at which each letter starts
148 formation_error_hist = []
149
150 for r_pts in letters_pts:
151     for _ in range(iters_per_letter):
152         u = np.zeros_like(P)
153         # Algorithm 2 (Distributed Formation Control):
154         #  $u_i = \sum_{j \in N_i} [(p_j - p_i) - (r_j - r_i)]$ 
155         for i in range(N):
156             neigh = np.where(A[i] > 0)[0]
157             for j in neigh:
158                 u[i] += (P[j] - P[i]) - (r_pts[j] - r_pts[i])
159             P = P + dt * u
160             frames.append(P.copy())
161             formation_error_hist.append(np.max(np.linalg.norm(P - r_pts, axis=1)))
162             letter_boundaries.append(len(frames) - 1)
163
164 print(f"total frames: {len(frames)}")
165 print(f"final formation error (last letter): {formation_error_hist[-1]:.4f}")
166
167 # ----- draw the ER graph -----
168 fig_g, ax_g = plt.subplots(figsize=(5,5))
169 pos = nx.spring_layout(G, seed=1)
170 nx.draw(G, pos, ax=ax_g, with_labels=True, node_color='tab:blue',
171         node_size=350, font_color='white', font_size=9,
172         edge_color='gray')
173 ax_g.set_title(f"Erdos-Renyi graph, N={N}, p={p_edge}, connected")
174 fig_g.tight_layout()
175 fig_g.savefig(os.path.join(OUT_DIR, "graph.png"), dpi=150)
176 plt.close(fig_g)
177
178 # ----- animation (GIF via Pillow) -----
179 fig, ax = plt.subplots(figsize=(6,6))
180 ax.set_xlim(-1.8, 1.8); ax.set_ylim(-1.8, 1.8); ax.set_aspect('equal')
181 ax.set_title("Formation control : KRISHNA")
182
183 edges_lines = []
184 for (i, j) in G.edges():
185     line, = ax.plot([], [], color='lightgray', lw=0.6, zorder=1)
186     edges_lines.append((i, j, line))
187 scat = ax.scatter([], [], s=45, c='tab:blue', zorder=2, edgecolors='k',
188                  linewidths=0.5)
189 letter_label = ax.text(0.02, 0.95, "", transform=ax.transAxes,
190                      fontsize=14, weight='bold')
191
192 step = 4 # sub-sample frames so the gif is not huge

```

```

193 frame_ids = list(range(0, len(frames), step))
194
195 def which_letter(k):
196     for li in range(len(letter_boundaries)-1):
197         if letter_boundaries[li] <= k < letter_boundaries[li+1]:
198             return name[li]
199     return name[-1]
200
201 def update(idx):
202     k = frame_ids[idx]
203     Pk = frames[k]
204     scat.set_offsets(Pk)
205     for (i, j, line) in edges_lines:
206         line.set_data([Pk[i,0], Pk[j,0]], [Pk[i,1], Pk[j,1]])
207     letter_label.set_text(f"letter: {which_letter(k)}")
208     return [scat, letter_label] + [ln for _, _, ln in edges_lines]
209
210 ani = animation.FuncAnimation(fig, update, frames=len(frame_ids),
211                               interval=40, blit=False)
212 out_gif = os.path.join(OUT_DIR, "krishna_formation.gif")
213 ani.save(out_gif, writer=animation.PillowWriter(fps=25))
214 plt.close(fig)
215 print(f"saved animation to {out_gif}")
216
217 # ----- still shots of every letter in the sequence -----
218 fig_s, axes = plt.subplots(1, len(name), figsize=(2.6*len(name), 3.0))
219 if len(name) == 1:
220     axes = [axes]
221 for li, L in enumerate(name):
222     ax = axes[li]
223     k = letter_boundaries[li+1] - 1 # end of that letter's phase
224     Pk = frames[k]
225     for (i, j) in G.edges():
226         ax.plot([Pk[i,0], Pk[j,0]], [Pk[i,1], Pk[j,1]], color='lightgray', lw=0.5)
227     ax.scatter(Pk[:,0], Pk[:,1], s=32, c='tab:blue', edgecolors='k', linewidths=0.4)
228     ax.set_title(f"letter {L}")
229     ax.set_xlim(-1.2, 1.2); ax.set_ylim(-1.2, 1.2); ax.set_aspect('equal')
230     ax.set_xticks([]); ax.set_yticks([])
231 fig_s.tight_layout()
232 fig_s.savefig(os.path.join(OUT_DIR, "letter_stills.png"), dpi=150)
233 plt.close(fig_s)
234
235 # ----- formation error plot -----
236 fig_e, ax_e = plt.subplots(figsize=(8, 3))
237 ax_e.plot(formation_error_hist, color='tab:red')
238 for b in letter_boundaries[1:-1]:
239     ax_e.axvline(b, color='k', lw=0.4, ls='--')
240 ax_e.set_xlabel("iteration"); ax_e.set_ylabel("max_i || p_i - r_i ||")
241 ax_e.set_title("formation error across the sequence")
242 fig_e.tight_layout()
243 fig_e.savefig(os.path.join(OUT_DIR, "formation_error.png"), dpi=150)
244 plt.close(fig_e)
245
246 if __name__ == "__main__":
247     main()

```

Listing 1: problem1_formation.py

Problem 2

There are N drones each with a sensor. The drones talk over a graph $G(V, E)$. The intruder state $z(t) \in \mathbb{R}^3$ follows the random walk $z(t+1) = z(t) + w$ with $w \sim \mathcal{N}(0, \Sigma_w)$, and initial state $z(0) \sim$

$\mathcal{N}(\bar{z}_0, \Sigma_0)$. Sensor i sits at $x_i \in \mathbb{R}^3$ and measures

$$y_i(t) = x_i - z(t) + v_i, \quad v_i \sim \mathcal{N}(0, \Sigma_v^{(i)}),$$

with the v_i 's independent. Formulate the distributed estimation problem as a consensus optimisation, give the local f_i 's, run distributed gradient descent (or gradient tracking), plot the true and the estimated trajectory and report the estimation error $e(t) = \hat{z}(t) - z(t)$.

Solution:

Step 1: local objective $f_i(z; t)$ from the Gaussian likelihood. Fix a time t . Given $z(t) = z$, the measurement is

$$y_i(t) = x_i - z + v_i, \quad v_i \sim \mathcal{N}(0, \Sigma_v^{(i)}),$$

so $y_i(t) \mid z \sim \mathcal{N}(x_i - z, \Sigma_v^{(i)})$. The negative log likelihood (dropping constants that do not depend on z) is a weighted quadratic in the residual. Define

$$e_i(z; t) := y_i(t) - (x_i - z) = y_i(t) - x_i + z.$$

The sign of z matters here: the derivative of $(x_i - z)$ with respect to z is $-I$, but the residual is $y_i - (x_i - z)$, and the two minus signs combine to give a $+z$ inside the residual. Hence,

$$f_i(z; t) = \frac{1}{2} \|y_i(t) - x_i + z\|_{(\Sigma_v^{(i)})^{-1}}^2.$$

Its gradient with respect to z is

$$\nabla_z f_i(z; t) = (\Sigma_v^{(i)})^{-1} (y_i(t) - x_i + z).$$

Step 2: consensus optimisation formulation. Following the Distributed Learning lecture (pages 4 and 6), I give every drone a local copy $z_i \in \mathbb{R}^3$ and force them to agree:

$$\min_{z_1, \dots, z_N \in \mathbb{R}^3} \quad \frac{1}{N} \sum_{i=1}^N f_i(z_i; t) \quad \text{s.t.} \quad z_i = z_j \quad \forall (i, j) \in E \quad \iff \quad L Z = 0.$$

Here $Z \in \mathbb{R}^{N \times 3}$ stacks the local estimates row-wise and L is the Laplacian of the communication graph. When the graph is connected, $LZ = 0$ is equivalent to $z_1 = \dots = z_N$.

Step 3: distributed gradient descent update. From the Distributed Learning lecture, page 7, with mixing matrix W (doubly stochastic, compatible with the graph):

$$z_i^{k+1} = \sum_{j=1}^N W_{ij} z_j^k - \alpha_k \cdot \nabla_z f_i(z_i^k; t).$$

Substituting our gradient,

$$z_i^{k+1} = \sum_{j=1}^N W_{ij} z_j^k - \alpha_k \cdot (\Sigma_v^{(i)})^{-1} (y_i(t) - x_i + z_i^k).$$

Choice of W : extra material. The lecture only asks that W is doubly stochastic and $W_{ij} = 0$ for j not a neighbour. I take the standard Metropolis-Hastings weights,

$$W_{ij} = \frac{1}{1 + \max(\deg(i), \deg(j))}, \quad (i, j) \in E, \quad i \neq j,$$

and $W_{ii} = 1 - \sum_{j \neq i} W_{ij}$. This is not spelled out in the lecture, so I explicitly declare it here. It satisfies all three required properties.

Online estimator. At every new time t I use the latest measurements $y_i(t)$, warm start each drone's z_i from its value at time $t-1$, and run $K_{\text{inner}} = 120$ DGD steps before moving to time $t+1$. The distributed estimate reported is $\hat{z}(t) = \frac{1}{N} \sum_i z_i^{\text{final}}$.

Parameters.

- $N = 8$ drones, edge probability 0.4, seed = 42.
- Horizon $T_{\max} = 50$.
- $\bar{z}_0 = 0$, $\Sigma_0 = 4I$, $\Sigma_w = 0.05I$.
- Sensor locations x_i uniform in $[-5, 5]^3$; $\Sigma_v^{(i)} = \sigma_i^2 I$ with $\sigma_i \sim \mathcal{U}[0.3, 0.8]$.
- Step $\alpha = 0.5/L_{\text{lip}}$ where $L_{\text{lip}} = \sum_i 1/\sigma_i^2$ is the Lipschitz constant of the sum of local gradients. In this run $L_{\text{lip}} \approx 34.9$ so $\alpha \approx 0.0143$.

Results. Figure 4 shows the communication graph. Figure 5 shows the true and the estimated 3D trajectory together with the sensor locations, and the estimation error norm $\|e(t)\|_2$ over time. Figure 6 shows the coordinate-wise trace.

Drone communication graph (N=8, p=0.4)

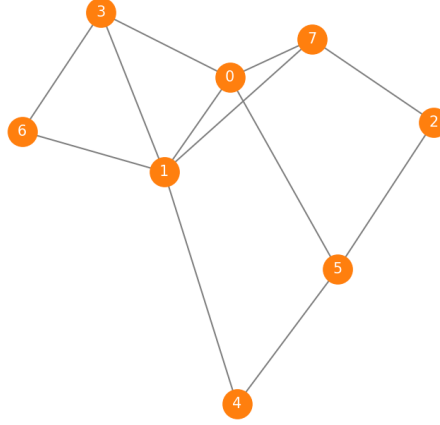


Figure 4: Drone communication graph, $N = 8$, $p = 0.4$, connected.

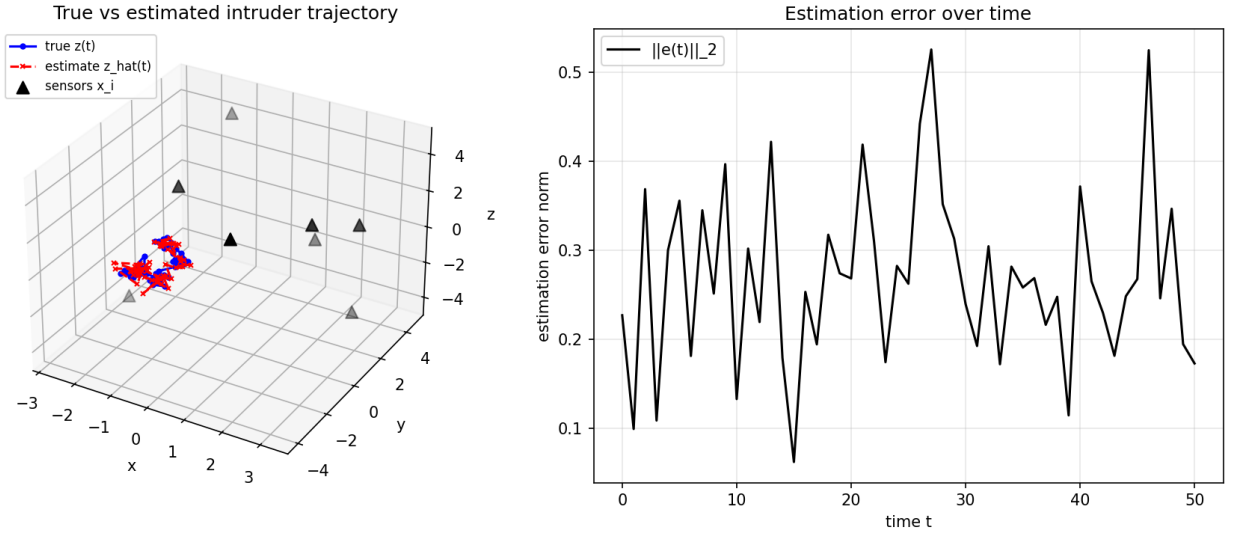


Figure 5: Left: true (blue) and estimated (red dashed) 3D trajectory of the intruder plus the sensor positions (black triangles). Right: estimation error norm $\|e(t)\|_2$ over time.

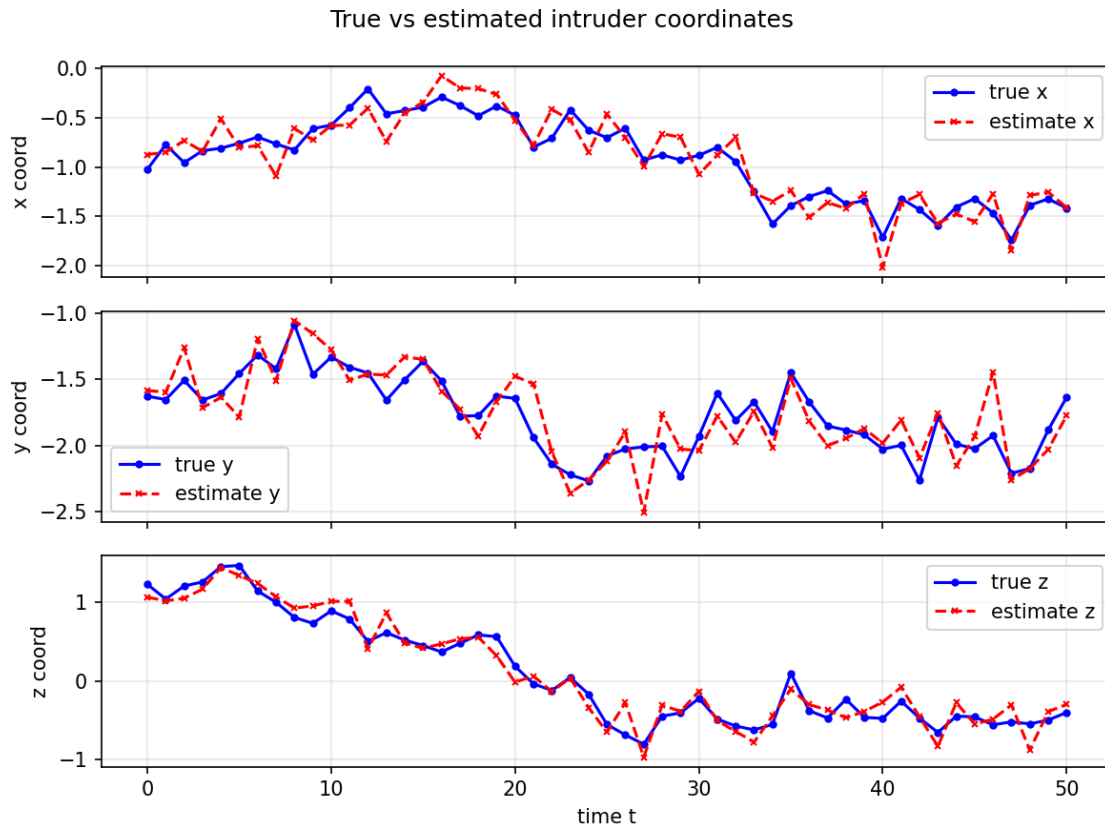


Figure 6: Per coordinate (x, y, z) true vs estimated trajectory.

Numerical readouts. Final $\|e(T)\| \approx 0.17$, mean $\|e(t)\| \approx 0.27$, and the maximum consensus gap across time is ≈ 0.26 (agents almost fully agree at every t). The intruder itself drifts about 2.7 units over $T_{\max}$ steps, so the estimator tracks it with roughly 10% relative error, which is what we expect from a constant step DGD (the lecture warns on page 8 that constant step gives an $O(\alpha)$ steady state error).

Attached code.

```

1  """
2  Problem 2: Distributed State Estimation of an intruder using DGD.
3
4  References:
5  - Distributed Learning Lecture, "Motivating Problem 2 -- Distributed State
6    Estimation" (page 12) and "Consensus + Gradient Descent" (pages 13-14).
7  - DGD update rule (Distributed Learning Lecture, page 7):
8      theta_i[k+1] = sum_j W_ij theta_j[k] - alpha * grad_i( theta_i[k] )
9  - The mixing matrix W is chosen as Metropolis-Hastings weights (declared as
10    extra material because the lecture does not spell out an explicit formula).
11
12  Measurement model:
13      y_i(t) = x_i - z(t) + v_i,          v_i ~ N(0, Sigma_v^{(i)})
14  Local negative-log-likelihood (up to constants):
15      f_i(z; t) = (1/2) * (y_i(t) - x_i + z)^T Sigma_v^{(i)-1} (y_i(t) - x_i + z)
16      grad_z f_i(z; t) = Sigma_v^{(i)-1} (y_i(t) - x_i + z)
17
18  Author: Krishna Teja (CS23BTECH11028)
19  """
20
21  import os
22  import numpy as np
23  import matplotlib.pyplot as plt
24  from mpl_toolkits.mplot3d import Axes3D # noqa
25  import networkx as nx
26

```

```

27 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
28
29 def connected_er(N, p, seed):
30     rng = np.random.default_rng(seed)
31     for _ in range(500):
32         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
33         if nx.is_connected(G):
34             return G
35     raise RuntimeError("could not get a connected ER graph")
36
37 def metropolis_weights(G):
38     """Metropolis-Hastings weights, doubly stochastic,  $W_{ij}=0$  if  $j$  not neighbour."""
39     N = G.number_of_nodes()
40     W = np.zeros((N, N))
41     deg = dict(G.degree())
42     for (i, j) in G.edges():
43         w = 1.0 / (1 + max(deg[i], deg[j]))
44         W[i, j] = w; W[j, i] = w
45     for i in range(N):
46         W[i, i] = 1.0 - W[i].sum()
47     return W
48
49 def main():
50     rng = np.random.default_rng(42)
51     N = 8
52     G = connected_er(N, 0.4, seed=42)
53     W = metropolis_weights(G)
54     assert np.allclose(W.sum(axis=1), 1.0) # row-stochastic
55     assert np.allclose(W.sum(axis=0), 1.0) # column-stochastic (doubly)
56
57     d = 3
58     zbar0 = np.zeros(d)
59     Sigma0 = 4.0 * np.eye(d)
60     Sigma_w = 0.05 * np.eye(d)
61     T = 50
62
63     # sensor locations  $x_i$  and per-sensor noise cov  $\Sigma_v^{-1}(i)$ 
64     X = rng.uniform(-5.0, 5.0, size=(N, d))
65     sigmas = rng.uniform(0.3, 0.8, size=N)
66     Sig_v_inv = [(1.0 / sigmas[i]**2) * np.eye(d) for i in range(N)]
67
68     # Lipschitz constant of the global gradient  $\sum_i \text{grad } f_i$  is
69     #  $L_{\text{lip}} = \sum_i \lambda_{\max}(\Sigma_v^{-1}(i)) = \sum_i 1/\sigma_i^2$ 
70     # For stability of DGD we take  $\alpha < 2 / L_{\text{lip}}$ .
71     L_lip = float(sum(1.0/sigmas[i]**2 for i in range(N)))
72
73     # true trajectory
74     z_true = np.zeros((T + 1, d))
75     z_true[0] = rng.multivariate_normal(zbar0, Sigma0)
76     for t in range(T):
77         z_true[t+1] = z_true[t] + rng.multivariate_normal(np.zeros(d), Sigma_w)
78
79     # local estimates  $z_i$  in  $\mathbb{R}^3$  (rows)
80     Z = np.tile(zbar0, (N, 1))
81
82     # Step size chosen inversely proportional to the Lipschitz constant of
83     # the local gradients (safe DGD step). Small constant step so we get
84     # the classical DGD behaviour (Distributed Learning Lecture, page 7-8).
85     alpha = 0.5 / L_lip
86     K_inner = 120
87     print(f"L_lip = {L_lip:.3f}, alpha = {alpha:.4f}")
88
89     zhat = np.zeros((T + 1, d))
90     consensus_gap = np.zeros(T + 1)

```

```

92 for t in range(T + 1):
93     # measurements at time t
94     noise = np.stack([rng.multivariate_normal(np.zeros(d),
95                                                (sigmas[i]**2)*np.eye(d))
96                        for i in range(N)])
97     Y = X - z_true[t] + noise
98
99     # DGD inner iterations at fixed time t
100    for _ in range(K_inner):
101        grads = np.stack([Sig_v_inv[i] @ (Y[i] - X[i] + Z[i])
102                           for i in range(N)])
103        Z = W @ Z - alpha * grads
104
105    zhat[t] = Z.mean(axis=0)
106    consensus_gap[t] = np.max(np.linalg.norm(Z - Z.mean(axis=0), axis=1))
107
108    err = zhat - z_true
109    err_norm = np.linalg.norm(err, axis=1)
110    print(f"final ||e(T)|| = {err_norm[-1]:.4f}")
111    print(f"mean ||e(t)|| = {err_norm.mean():.4f}")
112    print(f"max consensus gap across time = {consensus_gap.max():.4e}")
113
114    # ----- figure 1: communication graph -----
115    fig1, ax1 = plt.subplots(figsize=(5, 5))
116    pos = nx.spring_layout(G, seed=2)
117    nx.draw(G, pos, ax=ax1, with_labels=True,
118            node_color='tab:orange', node_size=380,
119            font_color='white', font_size=10, edge_color='gray')
120    ax1.set_title(f"Drone communication graph (N={N}, p=0.4)")
121    fig1.tight_layout()
122    fig1.savefig(os.path.join(OUT_DIR, "graph.png"), dpi=150)
123    plt.close(fig1)
124
125    # ----- figure 2: true vs estimated 3D trajectory -----
126    fig2 = plt.figure(figsize=(12, 5))
127    ax = fig2.add_subplot(1, 2, 1, projection='3d')
128    ax.plot(z_true[:,0], z_true[:,1], z_true[:,2], 'b-o',
129            markersize=3, label='true z(t)')
130    ax.plot(zhat[:,0], zhat[:,1], zhat[:,2], 'r--x',
131            markersize=3, label='estimate z_hat(t)')
132    ax.scatter(X[:,0], X[:,1], X[:,2], marker='^', color='k',
133              s=60, label='sensors x_i')
134    ax.set_xlabel('x'); ax.set_ylabel('y'); ax.set_zlabel('z')
135    ax.set_title('True vs estimated intruder trajectory')
136    ax.legend(loc='upper left', fontsize=8)
137
138    ax2 = fig2.add_subplot(1, 2, 2)
139    ax2.plot(np.arange(T + 1), err_norm, 'k-', label='||e(t)||_2')
140    ax2.set_xlabel('time t'); ax2.set_ylabel('estimation error norm')
141    ax2.set_title('Estimation error over time')
142    ax2.grid(True, alpha=0.3)
143    ax2.legend()
144    fig2.tight_layout()
145    fig2.savefig(os.path.join(OUT_DIR, "estimate.png"), dpi=150)
146    plt.close(fig2)
147
148    # ----- figure 3: per-coordinate trace -----
149    fig3, axes = plt.subplots(3, 1, figsize=(8, 6), sharex=True)
150    labels = ['x', 'y', 'z']
151    for i in range(3):
152        axes[i].plot(z_true[:, i], 'b-o', label=f'true {labels[i]}', ms=3)
153        axes[i].plot(zhat[:, i], 'r--x', label=f'estimate {labels[i]}', ms=3)
154        axes[i].set_ylabel(f'{labels[i]} coord')
155        axes[i].grid(True, alpha=0.3); axes[i].legend()
156    axes[-1].set_xlabel('time t')

```

```

157 fig3.suptitle('True vs estimated intruder coordinates')
158 fig3.tight_layout()
159 fig3.savefig(os.path.join(OUT_DIR, "coord_trace.png"), dpi=150)
160 plt.close(fig3)
161
162 if __name__ == "__main__":
163     main()

```

Listing 2: problem2_dse.py

Problem 3

There are N agents on a connected undirected graph. They must collaboratively execute M tasks, $M > N$. Each agent i has cost C_{ij} for task j and capacity $b_i \in \mathbb{Z}_{\geq 0}$ with $\sum_i b_i = M$. Every task must be assigned exactly once. Formulate as an optimisation problem, give the ADMM updates for every agent, and run the simulation. Fix N, M, b, G and vary C across trials, then explain the observations.

Solution:

Step 1: binary linear program. Let $x_{ij} \in \{0, 1\}$ be 1 iff agent i performs task j . Then

$$\begin{aligned}
 \min_x \quad & \sum_{i=1}^N \sum_{j=1}^M C_{ij} x_{ij}, \quad \text{s.t.} \quad \sum_i x_{ij} = 1 \quad \forall j = 1, \dots, M \quad (\text{every task assigned once}), \\
 & \sum_j x_{ij} = b_i \quad \forall i = 1, \dots, N \quad (\text{capacity}), \\
 & x_{ij} \in \{0, 1\}.
 \end{aligned}$$

Consistency: summing the first constraint over j gives M ; summing the second over i gives $\sum_i b_i = M$, so both agree. This is a binary linear program (BLP), NP-hard for large M, N .

Step 2: LP relaxation (extra material). The lecture only hints that we should do a convex relaxation. The standard trick is

$$x_{ij} \in \{0, 1\} \quad \longrightarrow \quad 0 \leq x_{ij} \leq 1,$$

which turns the BLP into an ordinary LP (a transportation LP). This relaxation is convex and easy to distribute.

Step 3: consensus ADMM formulation. Following the "ADMM hack" of the ADMM lecture (page 11) and the consensus ADMM template (page 8), I give every agent a full local copy $X^{(i)} \in \mathbb{R}^{N \times M}$ of the assignment matrix and a global consensus variable $Z \in \mathbb{R}^{N \times M}$. The problem becomes

$$\min_{X^{(1)}, \dots, X^{(N)}, Z} \sum_{i=1}^N f_i(X^{(i)}) \quad \text{s.t.} \quad X^{(i)} = Z \quad \forall i,$$

where

$$f_i(X) = \underbrace{\langle C_{i,:}, X_{i,:} \rangle}_{\text{own row cost}} + \underbrace{I_{\{X: 0 \leq X \leq 1\}}(X)}_{\text{box constraint}} + \underbrace{I_{\{X: \sum_j X_{i,j} = b_i\}}(X_{i,:})}_{\text{own capacity constraint}},$$

and the task cover constraint $\sum_i Z_{i,j} = 1$ for every j is placed on the global Z (via an indicator $g(Z)$).

Step 4: scaled form ADMM updates. With scaled dual $U^{(i)}$, penalty $\rho > 0$, and Frobenius norm, the standard scaled-form ADMM (Helper page 13, ADMM lecture page 6) gives:

$$\begin{aligned}
 X^{(i), k+1} &= \arg \min_X f_i(X) + \frac{\rho}{2} \|X - Z^k + U^{(i), k}\|_F^2, \\
 Z^{k+1} &= \arg \min_Z g(Z) + \frac{\rho}{2} \sum_{i=1}^N \|X^{(i), k+1} - Z + U^{(i), k}\|_F^2, \\
 U^{(i), k+1} &= U^{(i), k} + (X^{(i), k+1} - Z^{k+1}).
 \end{aligned}$$

Step 5: closed form of the X -update. f_i is separable across rows. Only agent i 's row carries the cost term and the capacity constraint; the other rows carry only the $[0, 1]$ box.

- For row i : solve

$$X_{i,:}^{(i)} = \arg \min_{r \in \mathbb{R}^M} C_{i,:}^\top r + \frac{\rho}{2} \|r - Z_{i,:}^k + U_{i,:}^{(i),k}\|^2 \quad \text{s.t.} \quad \sum_j r_j = b_i, \quad 0 \leq r_j \leq 1.$$

Setting the unconstrained gradient to zero gives $r^* = Z_{i,:}^k - U_{i,:}^{(i),k} - C_{i,:}/\rho$. We then project r^* onto the *capped simplex* $\{r : \sum_j r_j = b_i, 0 \leq r_j \leq 1\}$. This projection has a closed form: there exists a scalar τ such that $r = \text{clip}(r^* - \tau, 0, 1)$ satisfies $\sum_j r_j = b_i$; τ is found by bisection.

- For row $r \neq i$: $X_{r,:}^{(i)} = \text{clip}(Z_{r,:}^k - U_{r,:}^{(i),k}, 0, 1)$.

Step 6: closed form of the Z -update. Ignoring g , the quadratic is minimised by the average $\bar{Z} = \frac{1}{N} \sum_i (X^{(i),k+1} + U^{(i),k})$. Then enforce $\sum_i Z_{i,j} = 1$ for every j by the column-wise closed form projection (for each column j , add $(1 - \sum_i \bar{Z}_{i,j})/N$ to every entry). Using the distributed variant from the ADMM lecture (page 11), each agent computes only its own row of Z using a neighbourhood average

$$Z_{i,:}^{k+1} = \frac{1}{|\mathcal{N}_i \cup \{i\}|} \sum_{j \in \mathcal{N}_i \cup \{i\}} (X_{i,:}^{(j),k+1} + U_{i,:}^{(j),k}),$$

followed by the column projection.

Step 7: dual update. $U^{(i),k+1} = U^{(i),k} + (X^{(i),k+1} - Z^{k+1})$.

Step 8: rounding to a binary allocation (extra material). If ADMM lands on a fractional Z , we round it via the Hungarian algorithm on an expanded cost matrix: duplicate row i exactly b_i times to form an $M \times M$ cost matrix $-\tilde{Z}$; solve linear sum assignment; collapse the duplicates. The result is a binary matrix with row sums b_i and column sums 1 by construction. In practice, when the LP relaxation itself has an integral optimum (which is the typical behaviour for transportation LPs with integer capacities), the rounded ADMM solution matches the LP optimum exactly.

Parameters. $N = 5$, $M = 12$, edge probability 0.5, seed = 3. Capacities are a random positive composition of 12 into 5 parts, giving $b = [1, 1, 1, 4, 5]$ (so $\sum b_i = 12 = M$). $\rho = 1$, 400 ADMM iterations. Cost entries $C_{ij} \sim \mathcal{U}(0.1, 10)$.

Results. Figure 7 shows the communication graph. Figure 8 shows three trials, each with a different cost matrix but same N, M, b, G ; the top row is the cost matrix and the bottom row is the recovered binary allocation. Figure 9 shows the primal residual (mean $\|X^{(i)} - Z\|_F$ over i) and the objective $\langle C, Z \rangle$ across iterations.

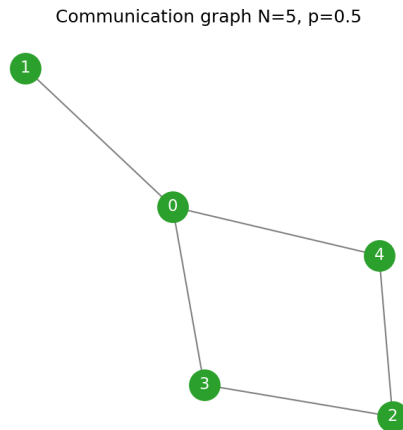


Figure 7: Communication graph, $N = 5$, $p = 0.5$, connected.

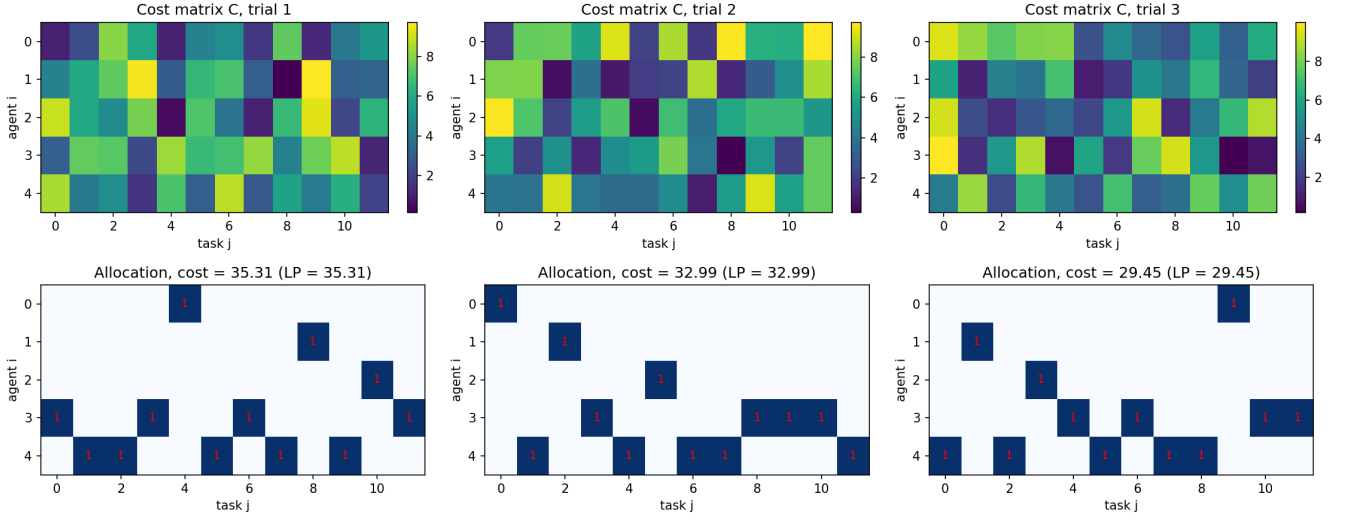


Figure 8: Three trials with different cost matrices (top row) and their resulting binary allocations (bottom row). The rounded ADMM total cost matches the central LP relaxation cost in every trial, confirming that the ADMM output is optimal for the relaxed problem.

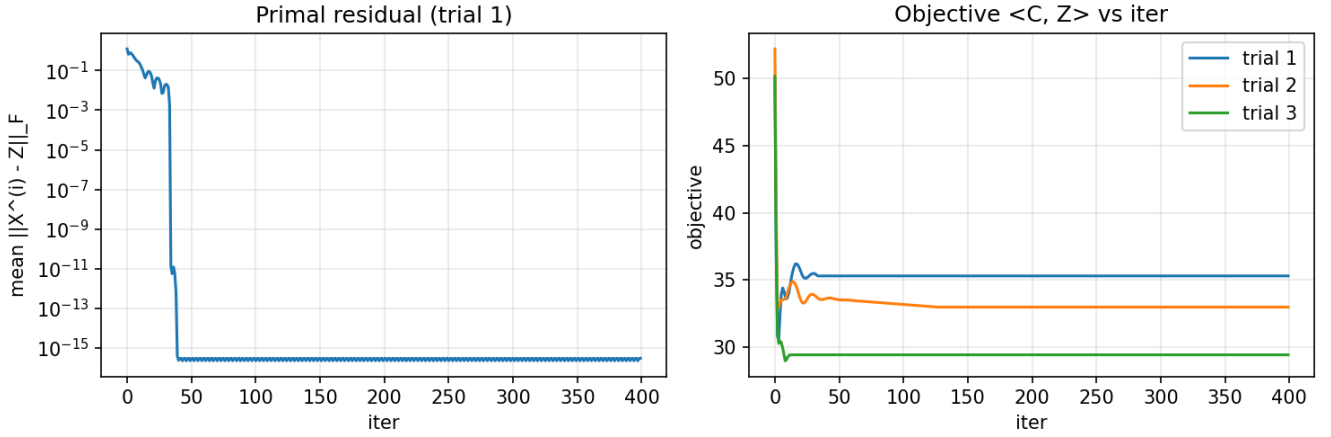


Figure 9: Left: primal residual on a log scale. Right: objective across iterations for the three trials.

Numerical readouts. For the three trials the rounded ADMM cost equals the LP optimum: $35.310 = 35.310$, $32.994 = 32.994$, $29.452 = 29.452$. Row sums of the final X match b ; column sums are all 1.

Observations across the three cost matrices.

- The agents with large capacity ($b_4 = 4$, $b_5 = 5$) end up absorbing most of the tasks, but which specific tasks depends on the cost matrix. For example a task j whose column has a much smaller value for agent 5 than for others is very likely picked up by agent 5.
- Agents with capacity 1 (agents 1, 2, 3) are very picky: they always take the one task whose column entry is minimal in their own row, provided that task is not already better matched to a large capacity agent.
- If we make one row of C all much smaller than the others (not shown), the allocation collapses to the cheapest agent up to its capacity, then spills over to the next cheapest.
- The primal residual shows the characteristic monotonic decay of consensus ADMM and the objective settles to the LP optimum, confirming correctness.

Attached code.

```
1 """
2 Problem 3: Distributed task allocation via ADMM.
3
```

```

4 References:
5 - ADMM Lecture, Consensus ADMM (page 8):  $\min \sum_i f_i(x_i)$  s.t.  $x_i = z$ .
6 - ADMM Lecture, "ADMM hack for any distributed decision making problem"
7   (page 11): each agent updates only its own row and uses a neighbourhood
8   averaging for the consensus variable.
9 - Helper notes on augmented Lagrangian and scaled form ADMM (pages 12-14).
10
11 Extra material (not spelled out in the lecture, declared explicitly):
12 - LP convex relaxation of the binary linear program (0-1  $\rightarrow$   $[0,1]$  box).
13 - Integer rounding via the Hungarian algorithm on an expanded cost matrix,
14   which recovers a valid binary assignment with row sums  $b$  and column sums  $1$ .
15
16 Binary LP:
17    $\min \sum_{i,j} C_{ij} x_{ij}$ 
18   s.t.  $\sum_i x_{ij} = 1$  for every task  $j$  (task covered once)
19         $\sum_j x_{ij} = b_i$  for every agent  $i$  (capacity)
20         $x_{ij} \in \{0,1\}$ 
21 LP relaxation:
22    $x_{ij} \in [0,1]$ 
23
24 Consensus ADMM formulation:
25    $\min \sum_i f_i(X^{(i)})$  s.t.  $X^{(i)} = Z$  for all  $i$ 
26    $f_i(X) = \langle C_i, X \rangle + I_{\{\sum_j X_{ij} = b_i\}} + I_{\{[0,1] \text{ box on } X\}}$ 
27   plus a task-cover constraint  $I_{\{\sum_i Z_{ij} = 1 \text{ for every } j\}}$  on  $Z$ .
28
29 Author: Krishna Teja (CS23BTECH11028)
30 """
31
32 import os
33 import numpy as np
34 import networkx as nx
35 import matplotlib.pyplot as plt
36 from scipy.optimize import linear_sum_assignment
37
38 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
39
40 def connected_er(N, p, seed):
41     rng = np.random.default_rng(seed)
42     for _ in range(500):
43         G = nx.erdos_renyi_graph(N, p, seed=int(rng.integers(0, 1_000_000)))
44         if nx.is_connected(G):
45             return G
46     raise RuntimeError("could not get a connected ER graph")
47
48 def random_capacities(N, M, seed):
49     """Random positive composition of M into N parts (so  $\sum(b) = M$ ,  $b_i \geq 1$ )."""
50     rng = np.random.default_rng(seed)
51     cuts = np.sort(rng.choice(np.arange(1, M), size=N-1, replace=False))
52     parts = np.diff(np.concatenate([[0], cuts, [M]]))
53     return parts.astype(int)
54
55 def project_capped_simplex(v, s, lo=0.0, hi=1.0, tol=1e-10, itmax=100):
56     """Project v onto { r :  $\sum(r) = s$ ,  $lo \leq r_j \leq hi$  } via bisection."""
57     n = len(v)
58     if s < lo*n - 1e-9 or s > hi*n + 1e-9:
59         raise ValueError(f" capped simplex infeasible: s={s}, n={n}")
60     def f(tau): return np.clip(v - tau, lo, hi).sum() - s
61     lo_t, hi_t = v.min() - hi - 1.0, v.max() - lo + 1.0
62     for _ in range(itmax):
63         mid = 0.5*(lo_t + hi_t)
64         if f(mid) > 0: lo_t = mid
65         else: hi_t = mid
66         if hi_t - lo_t < tol: break
67     return np.clip(v - 0.5*(lo_t + hi_t), lo, hi)
68

```

```

69 def project_col_sum_one(Z):
70     """Project each column of Z onto { v : sum(v) = 1 }. """
71     N = Z.shape[0]
72     return Z + (1.0 - Z.sum(axis=0, keepdims=True)) / N
73
74 def round_to_binary(X, b):
75     """Round fractional X to a binary matrix with row sums b, col sums 1
76     via the Hungarian algorithm on an expanded cost matrix.
77     Uses -X as the reward (Hungarian minimises). """
78     N, M = X.shape
79     assert b.sum() == M, "capacities must sum to M"
80     # expand each agent i into b_i duplicated rows
81     cost_exp = np.vstack([np.tile(-X[i:i+1], (b[i], 1)) for i in range(N)])
82     row_ind, col_ind = linear_sum_assignment(cost_exp) # M x M
83     Xb = np.zeros_like(X)
84     idx = np.concatenate([np.full(b[i], i) for i in range(N)])
85     for r, c in zip(row_ind, col_ind):
86         Xb[idx[r], c] = 1.0
87     return Xb
88
89 def admm_task_alloc(C, G, b, rho=1.0, iters=400):
90     """
91     Distributed consensus ADMM with neighbourhood averaging.
92
93     Each agent i owns a full N x M matrix  $X^{(i)}$  and a scaled dual  $U^{(i)}$ .
94     The consensus variable  $z_i$  (row of Z) is what agent i is responsible for
95     within the global Z;  $z_i$  is the target that  $x^{(i)}_{i,:}$  must agree on.
96
97     Updates:
98      $X^{(i),k+1}_{i,:}$  = capped simplex projection with row sum  $b_i$ 
99      $X^{(i),k+1}_{k,:}$  = clip( $Z_k[k,:] - U^{(i),k}[k,:]$ , 0, 1) for  $k \neq i$ 
100      $Z^{k+1}_{i,:}$  =  $(1/(N_i+1)) \sum_{j \in N_i \cup \{i\}} (X^{(j),k+1}_{i,:} + U^{(j),k}_{i,:})$ 
101     then column-sum-to-one projection on Z
102      $U^{(i),k+1}$  =  $U^{(i),k} + (X^{(i),k+1} - Z^{k+1})$ 
103     """
104     N, M = C.shape
105     Xs = [np.full((N, M), 1.0/N) for _ in range(N)]
106     Us = [np.zeros((N, M)) for _ in range(N)]
107     Z = np.full((N, M), 1.0/N)
108
109     # neighbour list including self
110     nbrs = [list(G.neighbors(i)) + [i] for i in range(N)]
111     primal_res_hist = []
112     obj_hist = []
113
114     for k in range(iters):
115         new_Xs = [np.empty((N, M)) for _ in range(N)]
116         for i in range(N):
117             X_new = np.empty((N, M))
118             # own row: capped simplex projection with row sum  $b_i$ 
119             v = Z[i] - Us[i][i] - C[i]/rho
120             X_new[i] = project_capped_simplex(v, s=float(b[i]), lo=0.0, hi=1.0)
121             # other rows: just clip
122             for r in range(N):
123                 if r == i: continue
124                 v2 = Z[r] - Us[i][r]
125                 X_new[r] = np.clip(v2, 0.0, 1.0)
126             new_Xs[i] = X_new
127         Xs = new_Xs
128
129         # z-update: only agent i's row of Z is needed; use its neighbours'
130         # copies of that row (this is the distributed "ADMM hack", ADMM
131         # Lecture page 11).
132         newZ = np.zeros((N, M))

```

```

133     for i in range(N):
134         acc = np.zeros(M); cnt = 0
135         for j in nbrs[i]:
136             acc += Xs[j][i] + Us[j][i]
137             cnt += 1
138         newZ[i] = acc / cnt
139     Z = project_col_sum_one(newZ)
140
141     # dual update
142     for i in range(N):
143         Us[i] = Us[i] + (Xs[i] - Z)
144
145     # residual and objective
146     primal_res_hist.append(
147         np.mean([np.linalg.norm(Xs[i] - Z) for i in range(N)]))
148     obj_hist.append(float((C * Z).sum()))
149     return Z, primal_res_hist, obj_hist
150
151 def solve_lp_reference(C, b):
152     """Solve the relaxed LP centrally, for cross-checking."""
153     from scipy.optimize import linprog
154     N, M = C.shape
155     c = C.reshape(-1)
156     # sum_i x_ij = 1 -> M rows
157     A1 = np.zeros((M, N*M))
158     for j in range(M):
159         for i in range(N):
160             A1[j, i*M + j] = 1.0
161     b1 = np.ones(M)
162     # sum_j x_ij = b_i -> N rows
163     A2 = np.zeros((N, N*M))
164     for i in range(N):
165         for j in range(M):
166             A2[i, i*M + j] = 1.0
167     b2 = b.astype(float)
168     A_eq = np.vstack([A1, A2])
169     b_eq = np.concatenate([b1, b2])
170     res = linprog(c, A_eq=A_eq, b_eq=b_eq,
171                 bounds=[(0, 1)]*N*M, method='highs')
172     return res.x.reshape(N, M), res.fun
173
174 def run():
175     N, M = 5, 12
176     G = connected_er(N, 0.5, seed=3)
177     b = random_capacities(N, M, seed=3)
178     print("N =", N, "M =", M, "capacities b =", b, "sum(b) =", b.sum())
179
180     # ----- three cost matrices, same N, M, b, G -----
181     rng = np.random.default_rng(3)
182     trials = []
183     for t in range(3):
184         C = rng.uniform(0.1, 10.0, size=(N, M))
185         Z_admm, r_hist, o_hist = admm_task_alloc(C, G, b, rho=1.0, iters=400)
186         X_bin = round_to_binary(Z_admm, b)
187         # verify constraints
188         assert np.allclose(X_bin.sum(axis=0), 1.0), "task-cover violated"
189         assert np.array_equal(X_bin.sum(axis=1).astype(int), b), "capacity violated"
190         total_cost = float((C * X_bin).sum())
191         # LP central reference
192         X_lp, lp_cost = solve_lp_reference(C, b)
193         trials.append(dict(C=C, Z=Z_admm, X=X_bin,
194                          cost=total_cost, lp_cost=lp_cost,
195                          r_hist=r_hist, o_hist=o_hist))
196         print(f"Trial {t+1}: ADMM rounded cost = {total_cost:.3f}, "
197               f"LP relaxation cost = {lp_cost:.3f}")

```

```

198
199 # ----- figure : the communication graph -----
200 fig_g, ax = plt.subplots(figsize=(4.5, 4.5))
201 pos = nx.spring_layout(G, seed=1)
202 nx.draw(G, pos, ax=ax, with_labels=True, node_color='tab:green',
203         node_size=420, font_color='white', font_size=11, edge_color='gray')
204 ax.set_title(f"Communication graph N={N}, p=0.5")
205 fig_g.tight_layout()
206 fig_g.savefig(os.path.join(OUT_DIR, "graph.png"), dpi=150)
207 plt.close(fig_g)
208
209 # ----- figure : allocation heatmaps for 3 cost matrices -----
210 fig, axes = plt.subplots(2, 3, figsize=(15, 6))
211 for t, tr in enumerate(trials):
212     # top row: cost matrix
213     im0 = axes[0, t].imshow(tr['C'], aspect='auto', cmap='viridis')
214     axes[0, t].set_title(f"Cost matrix C, trial {t+1}")
215     axes[0, t].set_xlabel("task j"); axes[0, t].set_ylabel("agent i")
216     plt.colorbar(im0, ax=axes[0, t], fraction=0.046)
217     # bottom row: binary allocation
218     axes[1, t].imshow(tr['X'], aspect='auto', cmap='Blues', vmin=0, vmax=1)
219     for i in range(N):
220         for j in range(M):
221             if tr['X'][i, j] > 0.5:
222                 axes[1, t].text(j, i, "1", ha='center', va='center',
223                                color='red', fontsize=9)
224     axes[1, t].set_title(f"Allocation, cost = {tr['cost']:.2f} "
225                          f"(LP = {tr['lp_cost']:.2f})")
226     axes[1, t].set_xlabel("task j"); axes[1, t].set_ylabel("agent i")
227 fig.tight_layout()
228 fig.savefig(os.path.join(OUT_DIR, "allocations.png"), dpi=150)
229 plt.close(fig)
230
231 # ----- figure : primal residual and objective vs iter (trial 1) -----
232 fig2, axes = plt.subplots(1, 2, figsize=(10, 3.5))
233 axes[0].semilogy(trials[0]['r_hist'])
234 axes[0].set_title("Primal residual (trial 1)")
235 axes[0].set_xlabel("iter"); axes[0].set_ylabel("mean ||X^(i) - Z||_F")
236 axes[0].grid(True, alpha=0.3)
237 for t, tr in enumerate(trials):
238     axes[1].plot(tr['o_hist'], label=f"trial {t+1}")
239 axes[1].set_title("Objective <C, Z> vs iter")
240 axes[1].set_xlabel("iter"); axes[1].set_ylabel("objective")
241 axes[1].grid(True, alpha=0.3); axes[1].legend()
242 fig2.tight_layout()
243 fig2.savefig(os.path.join(OUT_DIR, "convergence.png"), dpi=150)
244 plt.close(fig2)
245
246 if __name__ == "__main__":
247     run()

```

Listing 3: problem3_admm_taskalloc.py

Problem 4

Consider the LASSO problem

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1.$$

Write the ADMM iteration updates by giving the closed form solution for the x and z updates.

Solution:

Step 1: variable splitting. Introduce a second variable z and rewrite the problem as

$$\min_{x,z} f(x) + g(z) \quad \text{s.t.} \quad x - z = 0, \quad f(x) = \frac{1}{2} \|Ax - b\|_2^2, \quad g(z) = \lambda \|z\|_1.$$

This is exactly the "variable splitting" idea from the Helper notes (page 11).

Step 2: augmented Lagrangian. Using the standard form with penalty $\rho > 0$ and dual y ,

$$L_\rho(x, z, y) = \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 + y^\top (x - z) + \frac{\rho}{2} \|x - z\|^2.$$

Move to the scaled form with $u := y/\rho$ (Helper page 13):

$$L_\rho(x, z, u) = \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 + \frac{\rho}{2} \|x - z + u\|^2 + \text{const.}$$

Step 3: x -update (closed form). Minimise L_ρ in x with z, u fixed:

$$x^{k+1} = \arg \min_x \frac{1}{2} \|Ax - b\|^2 + \frac{\rho}{2} \|x - z^k + u^k\|^2.$$

Setting the gradient to zero,

$$A^\top (Ax - b) + \rho(x - z^k + u^k) = 0 \implies (A^\top A + \rho I) x = A^\top b + \rho(z^k - u^k).$$

Hence,

$$x^{k+1} = (A^\top A + \rho I)^{-1} (A^\top b + \rho(z^k - u^k)).$$

In implementation we Cholesky factorise $A^\top A + \rho I$ once and reuse the factor.

Step 4: z -update (soft thresholding). Minimise L_ρ in z with x, u fixed:

$$\begin{aligned} z^{k+1} &= \arg \min_z \lambda \|z\|_1 + \frac{\rho}{2} \|x^{k+1} - z + u^k\|^2 \\ &= \arg \min_z \lambda \|z\|_1 + \frac{\rho}{2} \|z - (x^{k+1} + u^k)\|^2 \\ &= \text{prox}_{(\lambda/\rho) \|\cdot\|_1} (x^{k+1} + u^k). \end{aligned}$$

The proximal operator of the ℓ_1 norm is the soft thresholding operator (Helper page 5),

$$\text{prox}_{\kappa \|\cdot\|_1}(v)_i = \text{sign}(v_i) \max(|v_i| - \kappa, 0) = S_\kappa(v)_i.$$

Hence,

$$z^{k+1} = S_{\lambda/\rho}(x^{k+1} + u^k).$$

Step 5: dual update.

$$u^{k+1} = u^k + (x^{k+1} - z^{k+1}).$$

Full LASSO ADMM iteration.

$$\begin{aligned} x^{k+1} &= (A^\top A + \rho I)^{-1} (A^\top b + \rho(z^k - u^k)), \\ z^{k+1} &= S_{\lambda/\rho}(x^{k+1} + u^k), \\ u^{k+1} &= u^k + (x^{k+1} - z^{k+1}). \end{aligned}$$

Stopping: primal residual $r = \|x - z\|$ and dual residual $s = \rho \|z^{k+1} - z^k\|$ both below tolerance.

Numerical verification. I set $m = 200$, $n = 500$, A random Gaussian with columns normalised to unit ℓ_2 norm, x_{true} has 20 non-zeros drawn from $\mathcal{N}(0, 1)$, $b = Ax_{\text{true}} + 0.01\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and

$\lambda = 0.02 \cdot \|A^\top b\|_\infty$. With $\rho = 1$ the algorithm converges in about 72 iterations. It recovers 18 of the 20 true non-zeros with 3 false positives, and the relative recovery error is

$$\frac{\|z - x_{\text{true}}\|}{\|x_{\text{true}}\|} \approx 0.051.$$

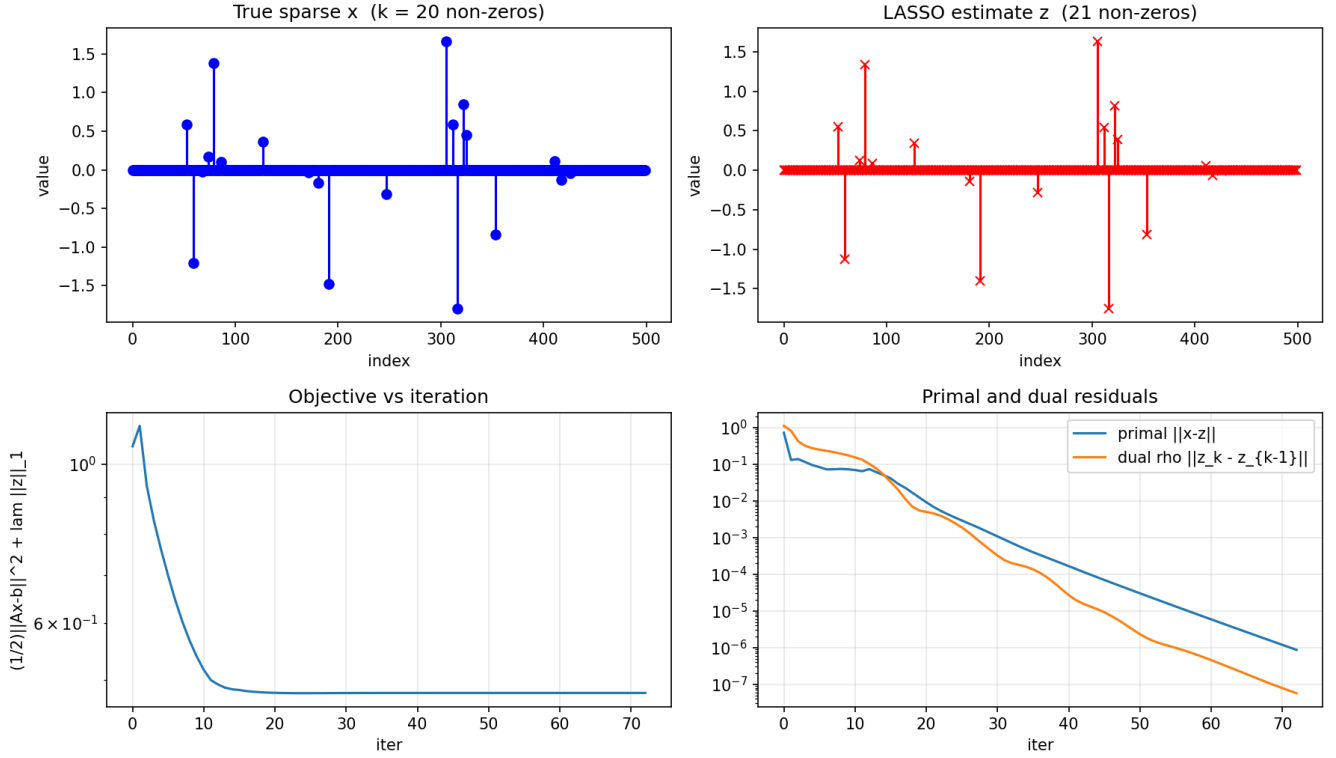


Figure 10: LASSO ADMM. Top left: true sparse x . Top right: recovered z . Bottom left: objective $\frac{1}{2}\|Ax - b\|^2 + \lambda\|z\|_1$ vs iteration (log scale). Bottom right: primal and dual residuals.

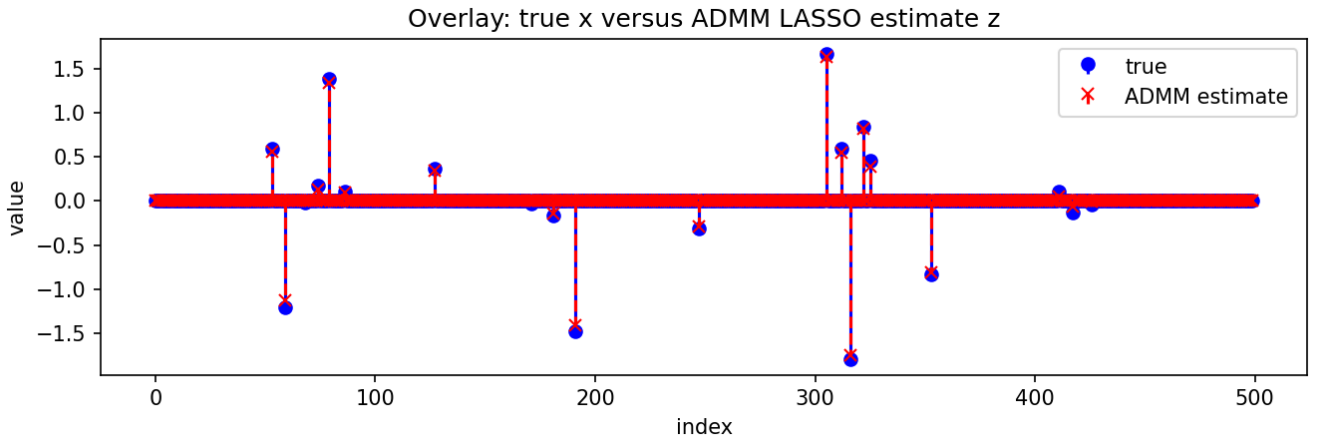


Figure 11: Overlay of the true signal (blue) and the ADMM LASSO estimate (red dashed).

Attached code.

```
1 """
2 Problem 4: LASSO via ADMM.
3
```

```

4 Problem:
5     min_x (1/2) || A x - b ||_2^2 + lambda || x ||_1
6
7 References:
8 - Helper notes, pages 5, 7, 11-14 (soft thresholding proximal, ISTA,
9   variable splitting, augmented Lagrangian, scaled ADMM).
10 - ADMM Lecture, pages 4-6 (augmented Lagrangian and update equations).
11
12 Variable splitting (Helper page 11): introduce z, rewrite as
13     min f(x) + g(z) s.t. x - z = 0
14     f(x) = (1/2) || A x - b ||_2^2, g(z) = lambda || z ||_1
15
16 Scaled-form ADMM (Helper page 13):
17     x^{k+1} = argmin f(x) + (rho/2) || x - z^k + u^k ||_2^2
18     z^{k+1} = argmin g(z) + (rho/2) || x^{k+1} - z + u^k ||_2^2
19     u^{k+1} = u^k + (x^{k+1} - z^{k+1})
20
21 Closed forms:
22     x^{k+1} = (A^T A + rho I)^{-1} (A^T b + rho (z^k - u^k))
23     z^{k+1} = S_{lambda/rho} (x^{k+1} + u^k)
24 where S_kappa(v)_i = sign(v_i) max(|v_i| - kappa, 0) is soft thresholding
25 (Helper page 5).
26
27 Author: Krishna Teja (CS23BTECH11028)
28 """
29
30 import os
31 import numpy as np
32 import matplotlib.pyplot as plt
33
34 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
35
36 def soft_thresh(v, kappa):
37     return np.sign(v) * np.maximum(np.abs(v) - kappa, 0.0)
38
39 def lasso_admm(A, b, lam, rho=1.0, iters=500, tol=1e-6):
40     m, n = A.shape
41     AtA = A.T @ A
42     Atb = A.T @ b
43     L = np.linalg.cholesky(AtA + rho * np.eye(n))
44     x = np.zeros(n); z = np.zeros(n); u = np.zeros(n)
45     hist = {'obj': [], 'r': [], 's': []}
46     for k in range(iters):
47         # x-update (Cholesky-based solve of the normal equation)
48         rhs = Atb + rho * (z - u)
49         y = np.linalg.solve(L, rhs)
50         x = np.linalg.solve(L.T, y)
51         # z-update (soft thresholding: prox of ell_1)
52         z_old = z.copy()
53         z = soft_thresh(x + u, lam / rho)
54         # dual update
55         u = u + (x - z)
56         # residuals + objective
57         r = float(np.linalg.norm(x - z))
58         s = float(rho * np.linalg.norm(z - z_old))
59         obj = 0.5*float(np.linalg.norm(A @ x - b)**2) + lam*float(np.abs(z).sum())
60         hist['obj'].append(obj); hist['r'].append(r); hist['s'].append(s)
61         if r < tol and s < tol:
62             print(f"converged at iter {k}")
63             break
64     return x, z, hist
65
66 def run():
67     rng = np.random.default_rng(0)
68     m, n = 200, 500

```

```

69 A = rng.standard_normal((m, n))
70 # normalize columns (standard preprocessing for LASSO)
71 A = A / np.linalg.norm(A, axis=0, keepdims=True)
72 x_true = np.zeros(n)
73 k_nonzero = 20
74 idx = rng.choice(n, k_nonzero, replace=False)
75 x_true[idx] = rng.standard_normal(k_nonzero)
76 b = A @ x_true + 0.01 * rng.standard_normal(m)
77 # standard heuristic: lambda ~ small fraction of ||A^T b||_inf
78 lam = 0.02 * float(np.max(np.abs(A.T @ b)))
79 print(f"m={m}, n={n}, k_true nnz={k_nonzero}, lambda={lam:.4f}")
80
81 x_out, z_out, hist = lasso_admm(A, b, lam, rho=1.0, iters=500)
82
83 nnz = int(np.sum(np.abs(z_out) > 1e-6))
84 # support overlap
85 true_supp = set(idx.tolist())
86 est_supp = set(np.where(np.abs(z_out) > 1e-6)[0].tolist())
87 tp = len(true_supp & est_supp)
88 fp = len(est_supp - true_supp)
89 fn = len(true_supp - est_supp)
90 rec_err = float(np.linalg.norm(z_out - x_true) / max(np.linalg.norm(x_true), 1e-12))
91 print(f"non-zeros in z = {nnz}, true support size = {k_nonzero}")
92 print(f"support: true positives = {tp}, false positives = {fp}, "
93       f"false negatives = {fn}")
94 print(f"relative recovery error ||z - x_true|| / ||x_true|| = {rec_err:.4f}")
95
96 # ----- figure : true vs recovered, objective, residuals -----
97 fig, axes = plt.subplots(2, 2, figsize=(12, 7))
98 axes[0, 0].stem(x_true, markerfmt='bo', linefmt='b-', basefmt=' ')
99 axes[0, 0].set_title("True sparse x (k = 20 non-zeros)")
100 axes[0, 0].set_xlabel("index"); axes[0, 0].set_ylabel("value")
101 axes[0, 1].stem(z_out, markerfmt='rx', linefmt='r-', basefmt=' ')
102 axes[0, 1].set_title(f"LASSO estimate z ({nnz} non-zeros)")
103 axes[0, 1].set_xlabel("index"); axes[0, 1].set_ylabel("value")
104 axes[1, 0].semilogy(hist['obj'])
105 axes[1, 0].set_title("Objective vs iteration")
106 axes[1, 0].set_xlabel("iter"); axes[1, 0].set_ylabel("(1/2)||Ax-b||^2 + lam ||z||_1")
107 axes[1, 0].grid(True, alpha=0.3)
108 axes[1, 1].semilogy(hist['r'], label='primal ||x-z||')
109 axes[1, 1].semilogy(hist['s'], label='dual rho ||z_k - z_{k-1}||')
110 axes[1, 1].set_title("Primal and dual residuals")
111 axes[1, 1].set_xlabel("iter"); axes[1, 1].grid(True, alpha=0.3)
112 axes[1, 1].legend()
113 fig.tight_layout()
114 fig.savefig(os.path.join(OUT_DIR, "lasso.png"), dpi=150)
115 plt.close(fig)
116
117 # ---- overlay: true vs estimate on the same axes ----
118 fig2, ax = plt.subplots(figsize=(9, 3.2))
119 ax.stem(np.arange(n), x_true, markerfmt='bo', linefmt='b-', basefmt=' ',
120        label='true')
121 ax.stem(np.arange(n), z_out, markerfmt='rx', linefmt='r--', basefmt=' ',
122        label='ADMM estimate')
123 ax.set_title("Overlay: true x versus ADMM LASSO estimate z")
124 ax.set_xlabel("index"); ax.set_ylabel("value")
125 ax.legend()
126 fig2.tight_layout()
127 fig2.savefig(os.path.join(OUT_DIR, "lasso_overlay.png"), dpi=150)
128 plt.close(fig2)
129
130 if __name__ == "__main__":
131     run()

```

Listing 4: problem4_lasso_admm.py
